

Task 1

Student Name : Kolganatov Ramazan

Student ID : 230103182

HOST CPU / SOC MODEL: 13th Gen Intel Core i9-13900HX

OPERATING SYSTEM & KERNEL: Windows 11 Home, Version 25H2, OS Build 26200, x64

Hardware Property:

CPU Model & Architecture: 13th Gen Intel(R) Core(TM) i9-13900HX, x86-64

Base & Max Boost Clock: 2.20 GHz / 5.40 GHz*

Physical Hardware Cores : 24

Logical Cores (SMT/Threads) : 32

L1 Data / Instruction Cache : 896 KB Data / 1280 KB Instruction total (*48/32 KB per P-core; 32/64 KB per E-core*)

L2 Cache (Per Core / Total) 32 MB total

L3 Shared Last-Level Cache 36 MB

SIMD Vector Extensions : SSE4.1, SSE4.2, AVX2

My computer supports SISD, SIMD, and MIMD execution. SISD is used when a single CPU thread executes one instruction stream on one data stream. SIMD is used when AVX2 vector instructions process multiple data elements with one instruction. MIMD is used when multiple CPU cores and threads execute different tasks simultaneously. My Intel Core i9-13900HX has 24 physical cores and 32 logical threads, so it can efficiently execute many parallel tasks.

Task 2 :

Threads N	Run 1	Run 2	Run 3	Avg Time TN	Speedup SN	Efficiency EN
1	2.675	2.733	2.509	2.639	1.000	100.0%
2	1.366	1.384	1.388	1.379	1.914	95.7%
4	0.819	0.866	0.832	0.839	3.146	78.6%
8	0.533	0.557	0.536	0.542	4.869	60.9%
16	0.448	0.463	0.475	0.462	5.712	35.7%
32	0.424	0.438	0.449	0.437	6.039	18.9%

Speedup begins to saturate at 16 workers. At 32 workers, speedup continues to increase slightly from 5.712× to 6.039×, but the improvement is much smaller than at lower worker counts. My CPU has 24 physical cores and 32 logical processors. As concurrency increases, scheduling overhead, cache contention, resource sharing between logical processors, and possible thermal/frequency effects reduce the benefit of additional workers.

```
Running with 2 processes...
Run 1: 1.366 sec
Run 2: 1.384 sec
Run 3: 1.388 sec
Average: 1.379 sec

Running with 4 processes...
Run 1: 0.819 sec
Run 2: 0.866 sec
Run 3: 0.832 sec
Average: 0.839 sec

Running with 8 processes...
Run 1: 0.533 sec
Run 2: 0.557 sec
Run 3: 0.536 sec
Average: 0.542 sec

Running with 16 processes...
Run 1: 0.448 sec
Run 2: 0.463 sec
Run 3: 0.475 sec
Average: 0.462 sec

Running with 32 processes...
Run 1: 0.424 sec
Run 2: 0.438 sec
Run 3: 0.449 sec
Average: 0.437 sec

=====
FINAL RESULTS
=====
N= 1   Average Time = 2.639 sec
N= 2   Average Time = 1.379 sec
N= 4   Average Time = 0.839 sec
N= 8   Average Time = 0.542 sec
N=16   Average Time = 0.462 sec
N=32   Average Time = 0.437 sec
```

Task 3 :

$$10 \times 1,000,000 = 10,000,000$$

Without Lock

Run	Measured Output	Error 10000000
#1	1,058,423	8,941,577
#2	1,031,900	8,968,100
#3	1,060,400	8,939,600
#4	1,014,900	8,985,100
#5	1,071,834	8,928,166
#6	1,048,100	8,951,900
#7	1,104,600	8,895,400
#8	1,002,100	8,997,900
#9	1,019,400	8,980,600
#10	1,034,800	8,965,200

With Lock

Across all 10 runs:

Actual = 10,000,000

1

The shared counter update is a Read-Modify-Write operation consisting of three steps: Load, Add, and Store. First, a thread loads the current value from shared memory. It then adds 1 to the value and stores the result back. If two threads read the same old value before either one stores its update, both threads calculate the same new value. When they store it, one update overwrites the other. Therefore, some increments are lost and the final counter becomes smaller than the expected 10,000,000.

2

Version	Average Time
Unlocked	705.84 ms
Locked	888.46 ms

Race-free synchronized code is substantially slower because every increment must acquire and release a lock. When multiple threads compete for the same lock, they cannot update the shared counter simultaneously and must wait for each other. This introduces synchronization overhead, contention, and additional scheduling costs. Therefore, although the lock guarantees the correct result, it reduces concurrency and increases execution time.

Task 4 :

1. Parallel Fraction Derivation

$T_1 = 2.639 \text{ s} \mid T_2 = 1.379 \text{ s}$

Calculated $p = 0.9545$

2. Amdahl Theoretical Speedup Ceiling

Sequential fraction $(1 - p) = 0.0455$

Max Theoretical Speedup ($N \rightarrow \infty$) = $21.98\times$

3. Amdahl 64-Core Projection

Projected Speedup on 64 Cores = $17.45\times$

4. Gustafson Scaled Speedup

Projected Scaled Speedup ($64\times$ Problem) = $61.09\times$

Amdahl's Law assumes a fixed problem size. As more cores are added, the parallel part becomes faster, but the sequential fraction remains a fixed bottleneck. Therefore, the maximum speedup is limited to approximately $21.98\times$, and the predicted speedup on 64 cores is about $17.45\times$.

Gustafson's Law assumes weak scaling, where the problem size increases as more cores are added. The sequential portion remains relatively small while the parallel portion grows with the number of cores. Therefore, Gustafson's model predicts a much larger scaled speedup of approximately $61.09\times$ on 64 cores.

In other words, Amdahl focuses on finishing the same problem faster, while Gustafson focuses on solving a larger problem in approximately the same amount of time.

